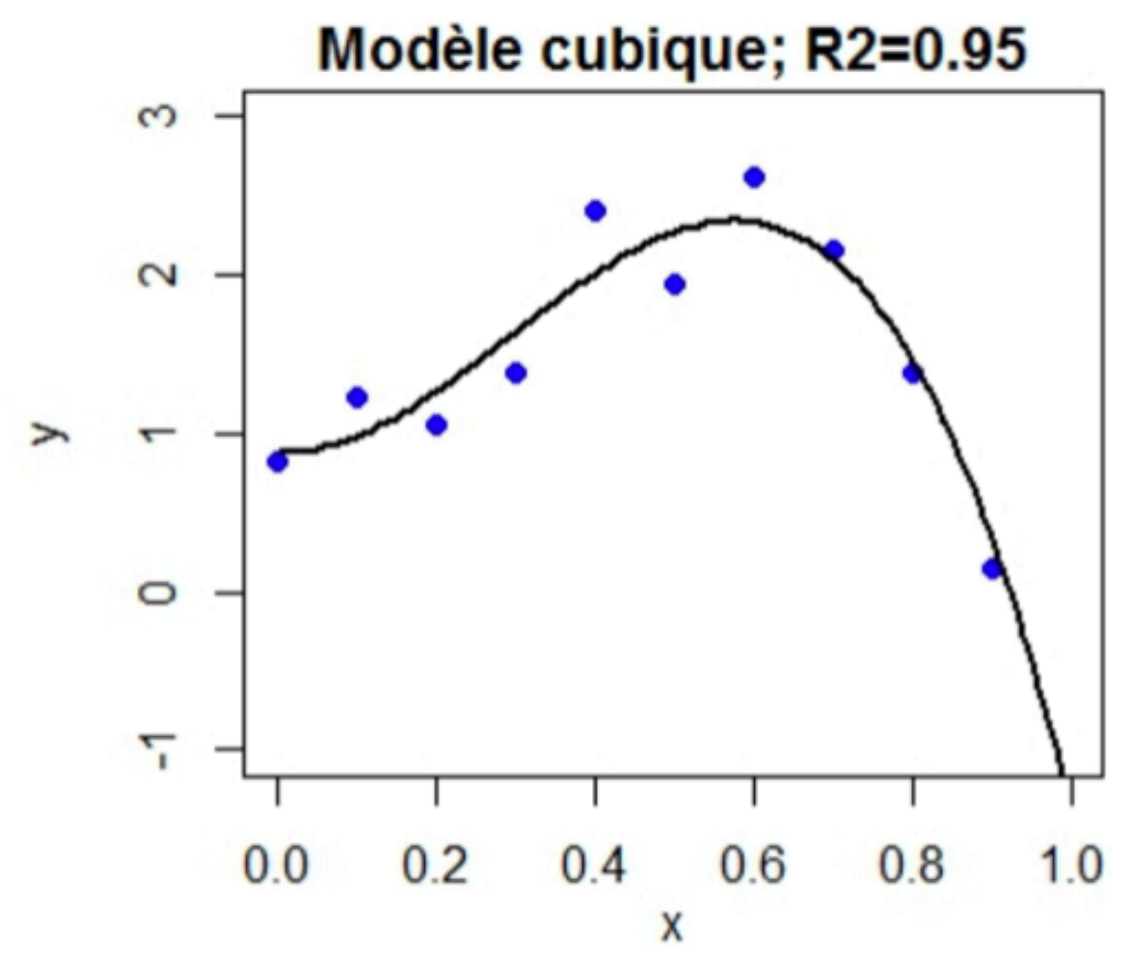
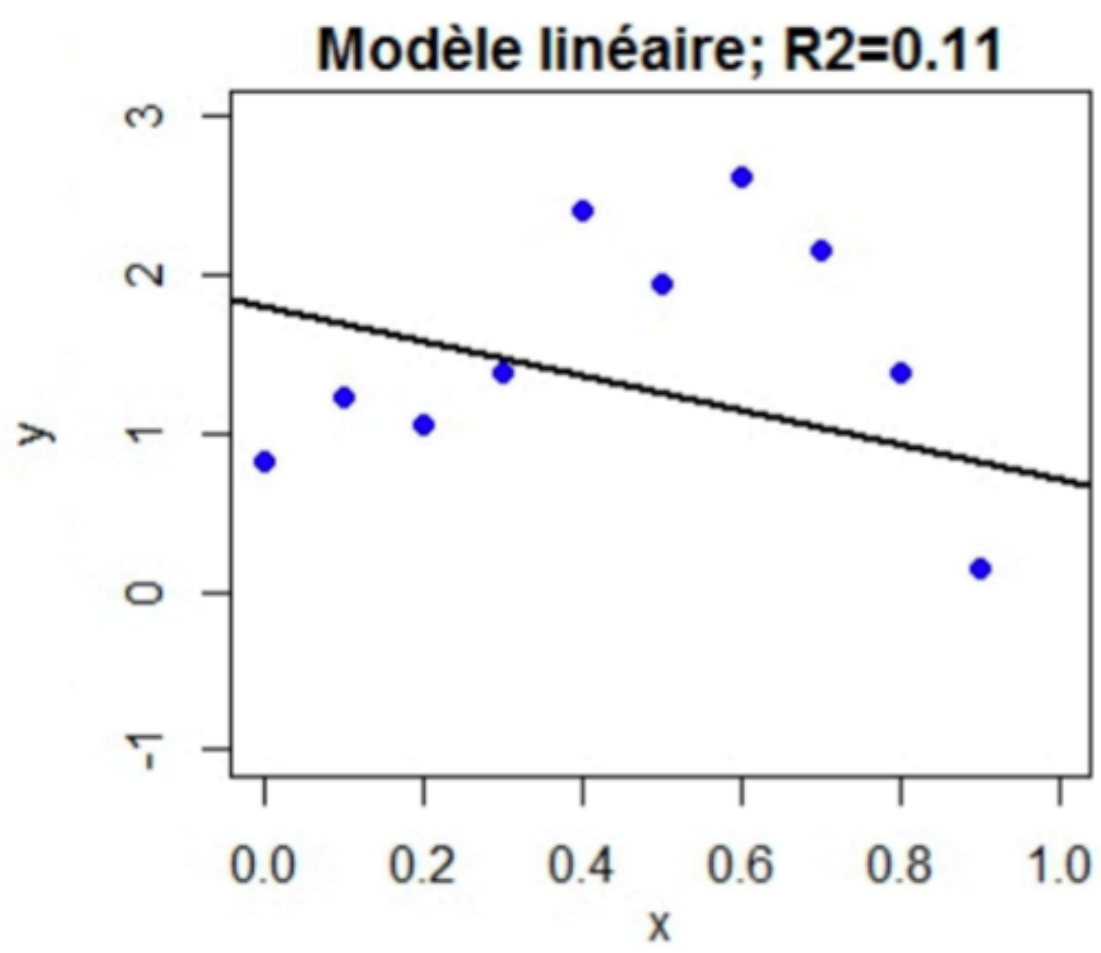


Fondements statistiques de l'apprentissage automatique

Chapitre 3 : Sélection de modèle en régression linéaire multiple

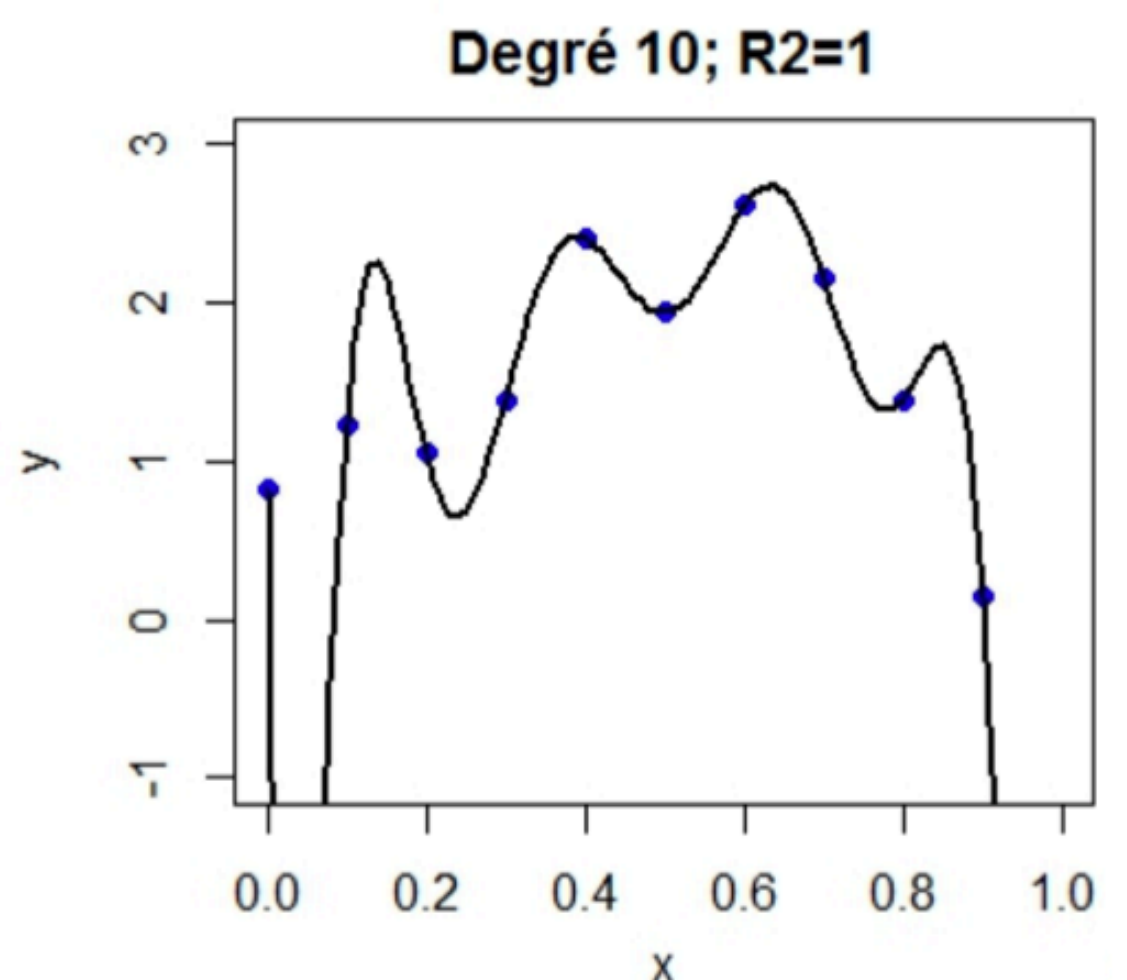
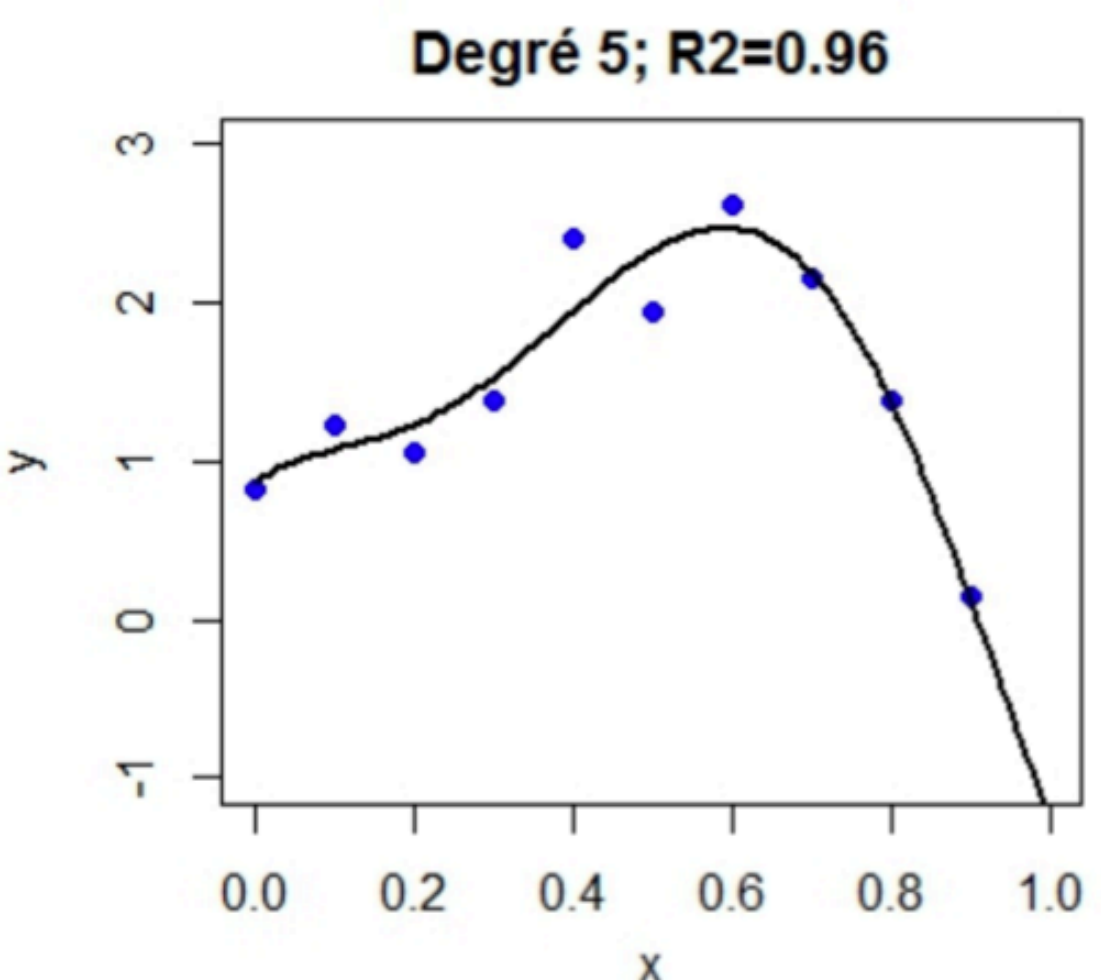
On approche les (x_i, y_i) à l'aide de polynômes de degrés K : $y_i = \beta_0 + \sum_{k=1}^K (\beta_k x_i^k) + \varepsilon$. Les résultats après estimation des β_k et le coefficient de détermination R^2 sont donnés ci-dessous pour $K = 1, 3, 5, 10$.

underfit



cooooo/
→ de nouvelles données colleraient bien au modèle.

hmm... ok

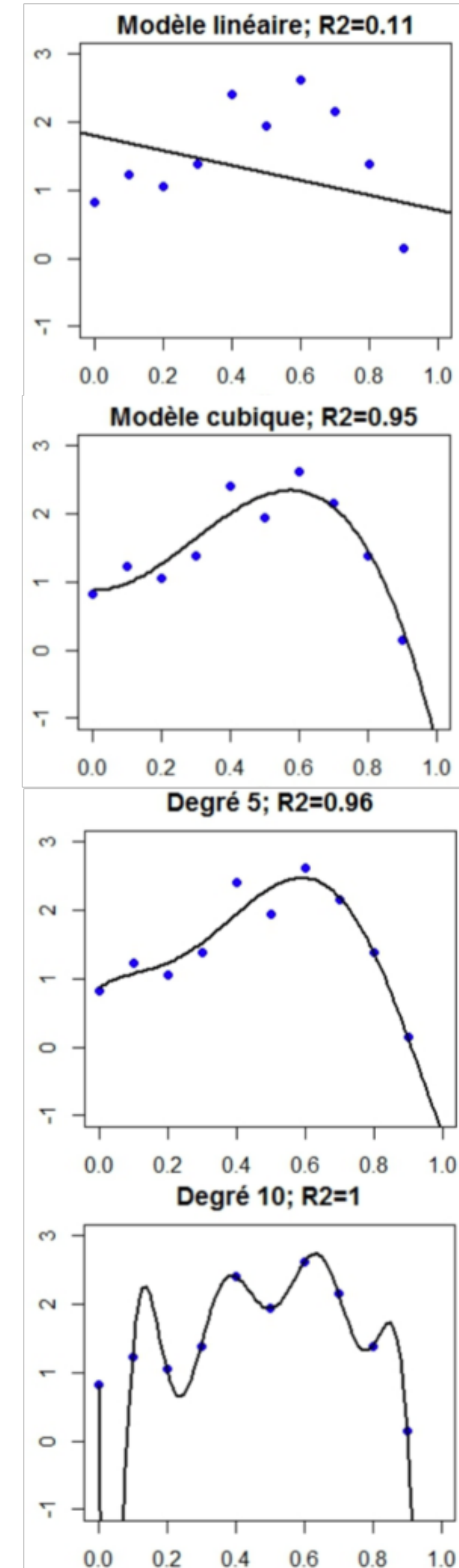


overfit / surapprentissage
Les LLMs surapprennent beaucoup.
C'est la cause des hallucinations : il fait coller le mauvais modèle à une question improbable.
Phénomène du sur-apprentissage !

On modélise souvent un problème de régression comme trouver la fonction f qui minimise le bruit ϵ_i sur n échantillons d'apprentissage (x_i, y_i) :

$$y_i = f(x_i) + \epsilon_i ,$$

où f est une fonction inconnue et ϵ_i suit une loi Normale de moyenne nulle et d'écart type σ . Le but de la regression est alors de trouver une fonction $\hat{f}$ qui approxime au mieux f . Ceci se fait en fixant d'abord un modèle (linéaire, polynôme, arbre de décision, réseau de neurones, ...) puis en apprenant ses q paramètres à partir de ce que l'on connaît, c'est à dire les (x_i, y_i) . Le problème qui émerge naturellement est le suivant : Comment simultanément estimer f au mieux et tenir le moins possible compte du bruit ϵ sachant que les deux sont inconnus ? C'est la question clé du compromis biais-variance.



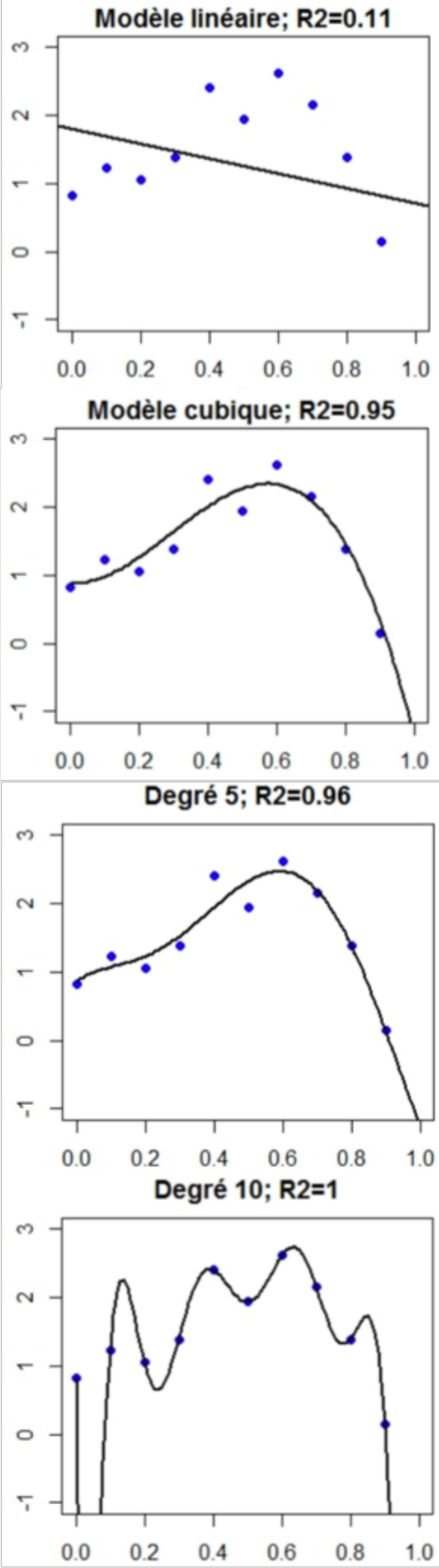
Plus formellement, on minimise l'esperance empirique de $(y - \hat{f}(x))^2$ sur les (x_i, y_i) , c'est à dire l'erreur au carré moyenne (Mean Squared Error – MSE). Elle peut être décomposée sous cette forme :

$$\mathbb{E} \left[(y - \hat{f}(x))^2 \right] = \underbrace{\mathbb{E} \left[\hat{f}(x) - f(x) \right]^2}_{\text{biais}[\hat{f}(x)]} + \underbrace{\mathbb{E} \left[\hat{f}(x)^2 \right] - \mathbb{E} \left[\hat{f}(x) \right]^2}_{\text{variance}[\hat{f}(x)]} + \sigma^2 \quad (3.1)$$

- Le terme de biais $\mathbb{E}[\hat{f}(x) - f(x)]^2$ représente à quel point le modèle $\hat{f}$ approxime la fonction inconnue f .
- Le terme de variance $\mathbb{E}[\hat{f}(x)^2] - \mathbb{E}[\hat{f}(x)]^2 = \text{Var}[\hat{f}(x)]$ représente le niveau de variabilité de $\hat{f}$, sans tenir compte de f .
- Le terme σ^2 représente enfin le niveau de bruit dans les données (x_i, y_i) , qui tout comme f est inconnu.

Pour une MSE (i.e. $\mathbb{E}[(y - \hat{f}(x))^2]$) donnée, un $\hat{f}$ représentera alors un compromis entre qualité d'approximation de f au niveau des observations $\{x_i\}_{i=1, \dots, n}$ et sa stabilité. Une trop grande qualité d'approximation au niveau des observations impliquera alors des fonctions $\hat{f}$ instables et ainsi moins généralisables en dehors des $\{x_i\}_{i=1, \dots, n}$ (sur-apprentissage). A contrario, des fonctions $\hat{f}$ trop stables captureront mal les relations entre les x_i et les y_i et auront de même un faible pouvoir prédictif.

Biais ↑
↓ Variance



Plus formellement, on minimise l'espérance empirique de $(y - \hat{f}(x))^2$ sur les (x_i, y_i) , c'est à dire l'erreur au carré moyenne (Mean Squared Error – MSE). Elle peut être décomposée sous cette forme :

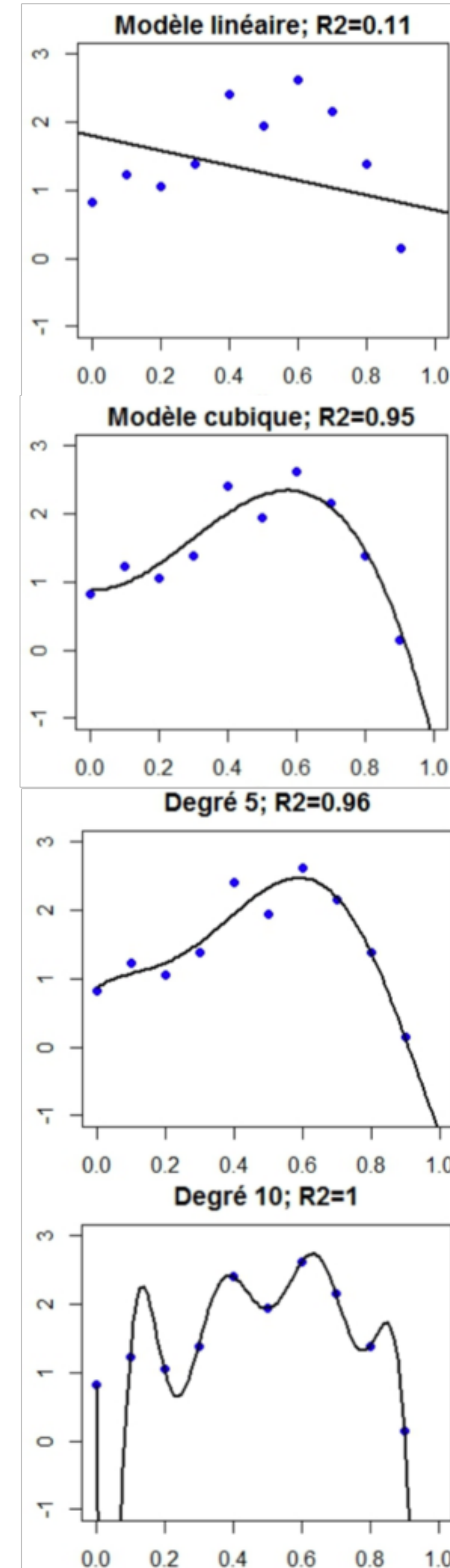
$$\mathbb{E} \left[(y - \hat{f}(x))^2 \right] = \underbrace{\mathbb{E} \left[\hat{f}(x) - f(x) \right]^2}_{\text{biais}[\hat{f}(x)]} + \underbrace{\mathbb{E} \left[\hat{f}(x)^2 \right] - \mathbb{E} \left[\hat{f}(x) \right]^2}_{\text{variance}[\hat{f}(x)]} + \sigma^2 \quad (3.1)$$

- Le terme de biais $\mathbb{E}[\hat{f}(x) - f(x)]^2$ représente à quel point le modèle $\hat{f}$ approxime la fonction inconnue f .
- Le terme de variance $\mathbb{E}[\hat{f}(x)^2] - \mathbb{E}[\hat{f}(x)]^2 = \text{Var}[\hat{f}(x)]$ représente le niveau de variabilité de $\hat{f}$, sans tenir compte de f .
- Le terme σ^2 représente enfin le niveau de bruit dans les données (x_i, y_i) , qui tout comme f est inconnu.

Trouver un bon compromis entre biais et variance pourra se faire en réduisant explicitement la dimension d'un modèle (Section 3.2) ou en régularisant l'estimation des paramètres d'un modèle (Section 3.3).

Biais

Variance



Considérons un modèle linéaire $\mathcal{M}$ à q variables $\mathbf{X}^{(j)}$, $j = 1, \dots, q$. Dans ce modèle $q < p$ et chaque $\mathbf{X}^{(j)}$ correspond à une des p variables observées $\mathbf{X}^k$, $k = 1, \dots, p$. Ce modèle s'écrit :

The image shows a handwritten matrix equation. On the left is a column vector of response variables $y_1, y_2, \dots, y_m$. This is equal to a matrix of predictors (each row starts with 1, followed by $x^{(1)}, x^{(2)}, \dots, x^{(q)}$) multiplied by a column vector of coefficients $\beta_0, \beta_{(1)}, \dots, \beta_{(q)}$, plus a column vector of error terms $\epsilon_1, \epsilon_2, \dots, \epsilon_m$.

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix} = \begin{pmatrix} 1 & x_1^{(1)} & x_1^{(2)} & \dots & x_1^{(q)} \\ 1 & x_2^{(1)} & x_2^{(2)} & \dots & x_2^{(q)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m^{(1)} & x_m^{(2)} & \dots & x_m^{(q)} \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_{(1)} \\ \vdots \\ \beta_{(q)} \end{pmatrix} + \begin{pmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_m \end{pmatrix}$$

La selection de modèle consiste à la fois à choisir les meilleur variables expli-
catives des y_i et à estimer les paramètres β_i optimaux. Nous développons dans
cette section plusieurs critères de sélection de modèle.

On a trop de données en entrée.
Il faut isoler et apprendre sur un sous - ensemble
de variables pertinentes.

Considérons un modèle linéaire $\mathcal{M}$ à q variables $\mathbf{X}^{(j)}$, $j = 1, \dots, q$. Dans ce modèle $q < p$ et chaque $\mathbf{X}^{(j)}$ correspond à une des p variables observées $\mathbf{X}^k$, $k = 1, \dots, p$. Ce modèle s'écrit :

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix} = \begin{pmatrix} 1 & x_1^{(1)} & x_1^{(2)} & \dots & x_1^{(q)} \\ 1 & x_2^{(1)} & x_2^{(2)} & \dots & x_2^{(q)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m^{(1)} & x_m^{(2)} & \dots & x_m^{(q)} \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_{(1)} \\ \vdots \\ \beta_{(q)} \end{pmatrix} + \begin{pmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_m \end{pmatrix}$$

La selection de modèle consiste à la fois à choisir les meilleur variables explicatives des y_i et à estimer les paramètres β_i optimaux. Nous développons dans cette section plusieurs critères de sélection de modèle.

old school

Critère C_p de Mallows

On rappelle que la somme des carrés des résidus $SSE = ||\mathbf{Y} - \hat{\mathbf{Y}}||^2 = ||e||^2$. On dénote alors la *mean square error* :

$$MSE = \frac{SSE}{n - p - 1},$$

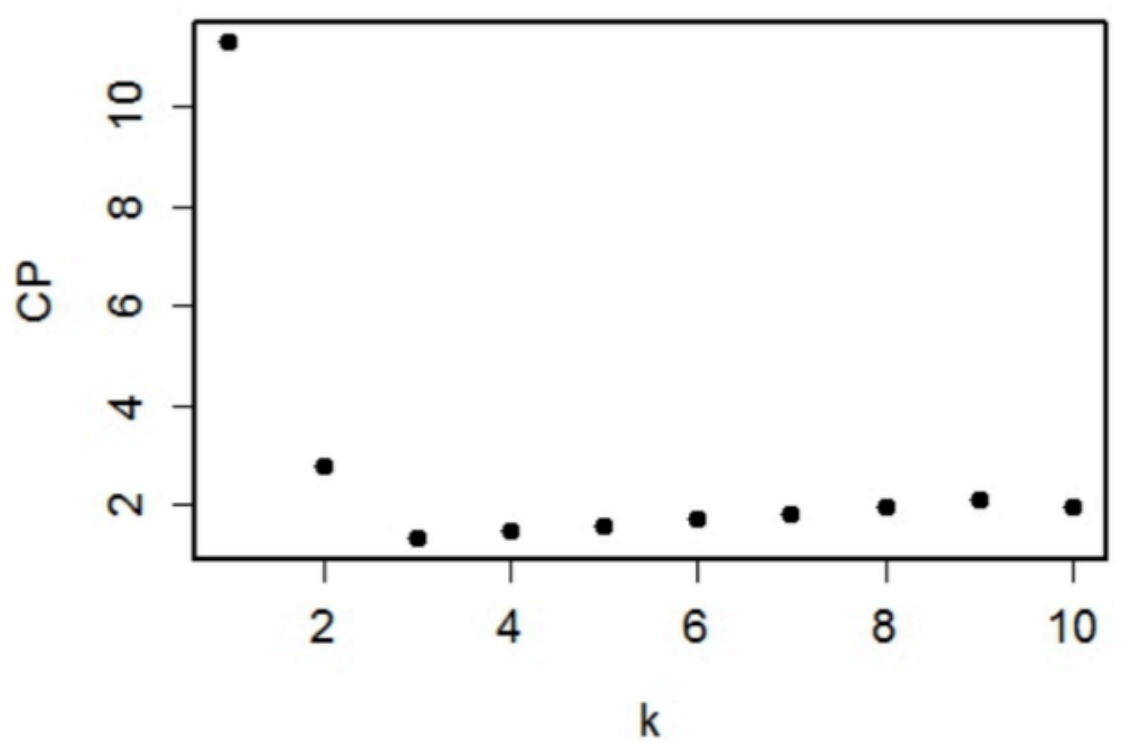
où $n - p - 1$ est le nombre de degrés de liberté du modèle compèt à p variables et n observations.

L'indicateur proposé par Mallows en 1973 pour evaluer la qualité d'un modèle donné $\mathcal{M}$ à q variables est alors

$$C_p = (n - (q + 1)) \frac{MSE_{\mathcal{M}}}{MSE} - (n - 2(q + 1))$$

où $MSE_{\mathcal{M}}$ est la MSE calculée pour le modèle $\mathcal{M}$.

Il est alors d'usage de rechercher un modèle qui minimise le C_p . Ceci revient à considérer que le "vrai" modèle complet est moins fiable qu'un modèle réduit donc biaisé mais d'estimation plus précise. A qualité de modèle constant $MSE_{\mathcal{M}}/MSE$, plus q est faible, plus C_p est faible. Par contre si l'erreur du modèle $\mathcal{M}$ augmente à q fixé, C_p augmente. Voila ci-dessous l'évolution de C_p en fonction de K dans l'exemple introductif du chapitre. Ici, le meilleur modèle contient $q = 3$ variables.



Considérons un modèle linéaire $\mathcal{M}$ à q variables $\mathbf{X}^{(j)}$, $j = 1, \dots, q$. Dans ce modèle $q < p$ et chaque $\mathbf{X}^{(j)}$ correspond à une des p variables observées $\mathbf{X}^k$, $k = 1, \dots, p$. Ce modèle s'écrit :


$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix} = \begin{pmatrix} 1 & x_1^{(1)} & x_1^{(2)} & \dots & x_1^{(q)} \\ 1 & x_2^{(1)} & x_2^{(2)} & \dots & x_2^{(q)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m^{(1)} & x_m^{(2)} & \dots & x_m^{(q)} \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_{(1)} \\ \vdots \\ \beta_{(q)} \end{pmatrix} + \begin{pmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_m \end{pmatrix}$$

La selection de modèle consiste à la fois à choisir les meilleur variables expli-
catives des y_i et à estimer les paramètres β_i optimaux. Nous développons dans
cette section plusieurs critères de sélection de modèle.

id (un peu ancienne) : un modèle avec moins de variable
se généralise mieux.

Critères AIC, BIC et PRESS

Dans le cas du modèle linéaire, et si la variance des observations est sup-
posée connue, le critère AIC (Akaike's Information criterium) est équivalent au
critère C_p de Mallows. Le critère BIC (Bayesian Information Criterium) est une
extension d'AIC dans lequel le terme de pénalité est plus important. Le PRESS
(somme des erreurs quadratiques) de Allen (1974) est l'introduction historique
de la validation croisée ou leave-one-out (loo). Ces critères peuvent être résumés
par :

- AIC : $AIC(\mathcal{M}) = n \log MSE_{\mathcal{M}} + 2(q + 1)$
- BIC : $AIC(\mathcal{M}) = n \log (MSE_{\mathcal{M}}) + \log (n)(q + 1)$
- PRESS : On désigne par $\widehat{y_{(-i)j}}$ la prévision de y_j calculée sans tenir
compte de la ième observation lors de l'estimation des paramètres alors :
 $PRESS = \sum_{i=1}^n (y_i - \widehat{y_{(-i)i}})^2$

et permettent de comparer les capacités prédictives de différents modèles.

Considérons un modèle linéaire $\mathcal{M}$ à q variables $\mathbf{X}^{(j)}$, $j = 1, \dots, q$. Dans ce modèle $q < p$ et chaque $\mathbf{X}^{(j)}$ correspond à une des p variables observées $\mathbf{X}^k$, $k = 1, \dots, p$. Ce modèle s'écrit :


$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix} = \begin{pmatrix} 1 & x_1^{(1)} & x_1^{(2)} & \dots & x_1^{(q)} \\ 1 & x_2^{(1)} & x_2^{(2)} & \dots & x_2^{(q)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m^{(1)} & x_m^{(2)} & \dots & x_m^{(q)} \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_{(1)} \\ \vdots \\ \beta_{(q)} \end{pmatrix} + \begin{pmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_m \end{pmatrix}$$

La selection de modèle consiste à la fois à choisir les meilleur variables explicatives des y_i et à estimer les paramètres β_i optimaux. Nous développons dans cette section plusieurs critères de sélection de modèle.

étape 0

Apprend avec v_{s_0} ($s_0 = 1$)

Apprend avec v_1

$\vdots$

v_p

Ajoute v_{s_1} qui réduit le plus msge.

étape n

Apprend avec $(v_{s_0}, v_{s_1}, \dots, v_{s_n})$

Apprend avec toutes sont elles.

Ajoute $v_{s_{n+1}}$.

Algorithmes de sélection de variables

Dans le cas général les variables ne sont pas pré-ordonnées par importance. C'est d'ailleurs le cas le plus courant en pratique ! Lorsque p est grand, il n'est pas raisonnable d'explorer les 2^p modèles possibles afin de sélectionner le meilleur au sens de l'un des critères ci-dessus. Différentes stratégies existent pour explorer efficacement les modèles possibles. Elles doivent être choisies en fonction de l'objectif recherché, de la valeur de p et des moyens de calcul disponibles. Deux types d'algorithmes sont résumés ci-dessous par ordre croissant de temps de calcul nécessaire, c'est-à-dire par nombre croissant de modèles considérés explorés parmi les 2^p et ainsi par capacité croissante d'optimalité.

Sélection (forward) A l'état initial $q = 1$ et toutes les p variables sont testées. La variable qui permet de réduire au mieux le critère du modèle obtenu est sélectionnée, on la dénote (1). On teste alors si une des $p - 1$ variables restantes améliore la qualité du modèle avec $q = 2$ et (1) déjà sélectionné... et ainsi de suite. La procédure s'arrête lorsque toutes les variables sont introduites ou lorsque le critère ne décroît plus.

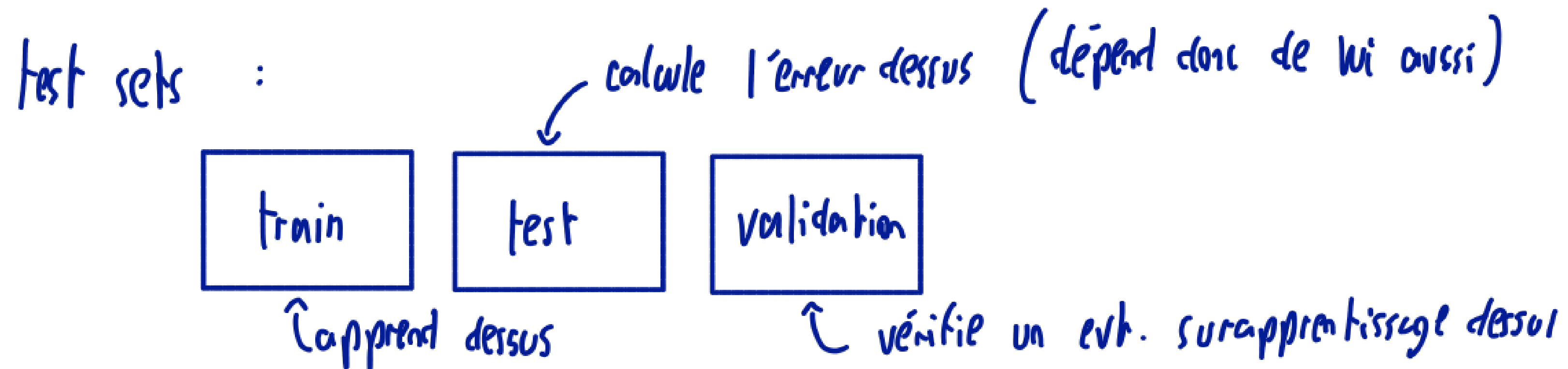
Elimination (backward) L'algorithme démarre cette fois du modèle complet. À chaque étape, la variable dont l'élimination conduit la valeur du critère la plus faible est supprimée. La procédure s'arrête lorsque la valeur du critère ne décroît plus.

Mixte (stepwise) Cet algorithme introduit une étape d'élimination de variable après chaque étape de sélection afin de retirer du modèle d'éventuels variables qui seraient devenues moins indispensables du fait de la présence de celles nouvellement introduites.

fonctionnent jusqu'en dimension ~100

Le monde moderne : p grand

Les méthodes de régression régularisée sont à utiliser quand le problème est mal conditionné, et typiquement quand le nombre d'observations n est plus petit que la dimension des observations p . Ce cas est très courant en pratique, par exemple quand chaque observation coûte cher à obtenir mais est en très grande dimension, comme c'est le cas en génomique ou dans de nombreuses applications industrielles.



3.3.1 Régression ridge

Modèle et estimation

L'estimateur ridge est défini par un critère des moindres carrés, avec une pénalité de type $\mathbb{L}^2$ par :

$$\hat{\beta}_{ridge} = \arg \min_{\beta \in \mathbb{R}^{p+1}} \left(\sum_{i=1}^n (Y_i - \sum_{j=0}^p X_i^{(j)} \beta_j)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right)$$

où λ est un paramètre positif. Notez que le paramètre β_0 n'est pas pénalisé.

En supposant $\mathbf{X}$ et $\mathbf{Y}$ centrés, l'estimateur ridge est obtenu en résolvant les équations normales qui s'expriment sous la forme :

$$\mathbf{X}'\mathbf{Y} = (\mathbf{X}'\mathbf{X} + \lambda \mathbf{I}_p) \beta$$

Conduisant à :

$$\hat{\beta}_{ridge} = (\mathbf{X}'\mathbf{X} + \lambda \mathbf{I}_p)^{-1} \mathbf{X}'\mathbf{Y}$$

La solution est donc explicite et linéaire en $\mathbf{Y}$. Remarquons alors que :

- $\mathbf{X}'\mathbf{X}$ est une matrice symétrique positive, *i.e.* pour tout vecteur $\mathbf{u}$ de $\mathbb{R}^p$: $\mathbf{u}'(\mathbf{X}'\mathbf{X})\mathbf{u} \geq 0$. Il en résulte que pour tout $\lambda > 0$, $\mathbf{X}'\mathbf{X} + \lambda \mathbf{I}_p$ est inversible.
- La constante β_0 n'intervient pas dans la pénalité, sinon, le choix de l'origine pour $\mathbf{Y}$ aurait une influence sur l'estimation de l'ensemble des paramètres. Alors : $\hat{\beta}_0 = \bar{\mathbf{Y}}$; ajouter une constante à $\mathbf{Y}$ ne modifie pas les $\hat{\beta}_j$ pour $j \geq 1$.

il faut que λ ne soit pas négligeable devant les coeffs diagonaux.

3.3.1 Régression ridge

Modèle et estimation

L'estimateur ridge est défini par un critère des moindres carrés, avec une pénalité de type $\mathbb{L}^2$ par :

$$\hat{\beta}_{ridge} = \arg \min_{\beta \in \mathbb{R}^p} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_{i1} - \dots - \beta_p x_{ip})^2 + \lambda \sum_{j=1}^p \beta_j^2$$

où λ est un paramètre positif.
En supposant $\mathbf{X}$ et $\mathbf{Y}$ les matrices des données, les équations normales qui s'écrivent :

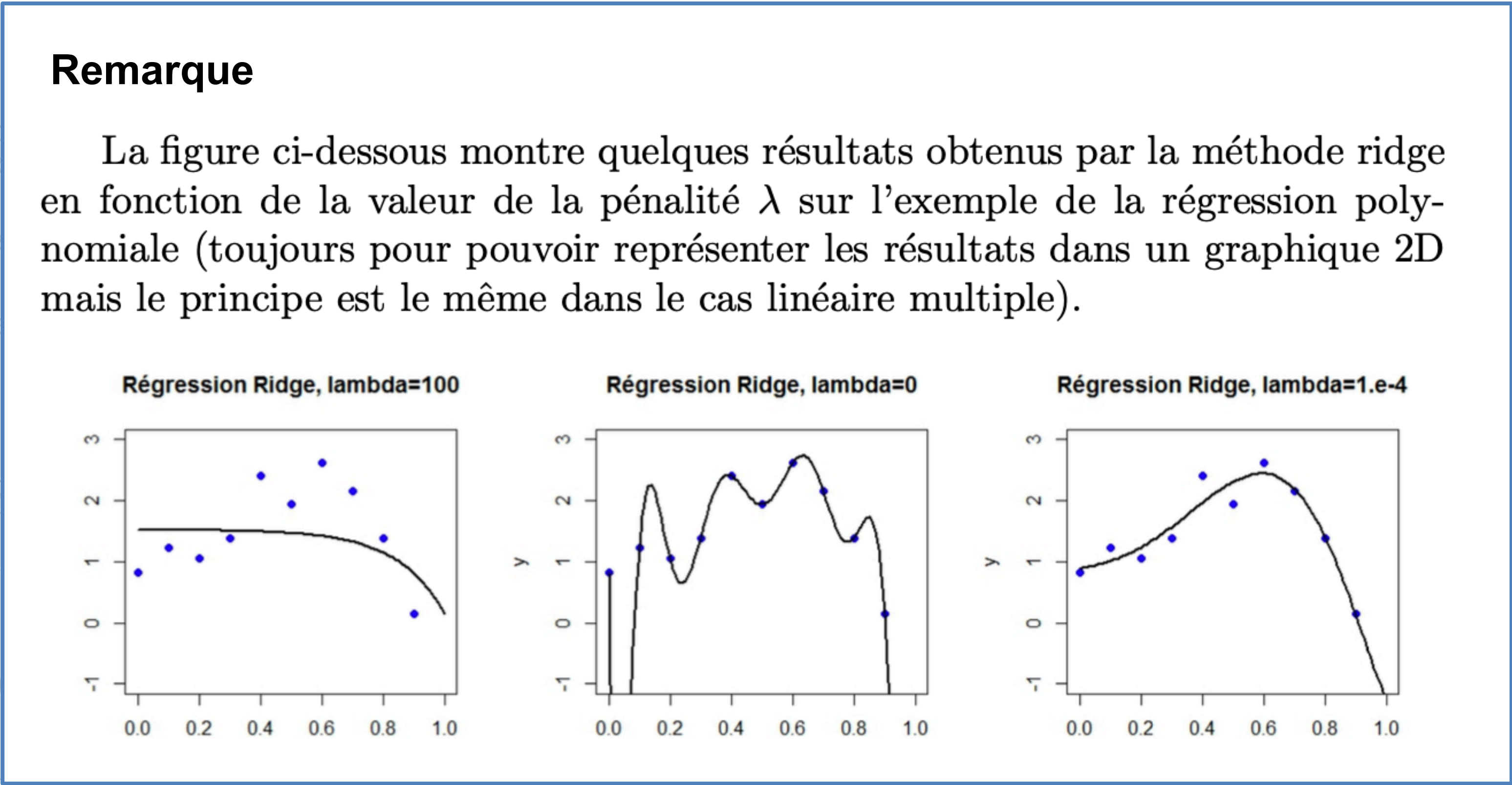
Conduisant à :

La solution est donc explicite :

- $\mathbf{X}'\mathbf{X}$ est une matrice $p \times p$ sur $\mathbb{R}^p$: $\mathbf{u}'(\mathbf{X}'\mathbf{X})\mathbf{u} \geq 0$ et est inversible.
- La constante β_0 n'influence pas l'estimation des autres paramètres. Alors : $\hat{\beta}_0 = \bar{Y}$; ajouter une constante à $\mathbf{Y}$ ne modifie pas les $\hat{\beta}_j$ pour $j \geq 1$.

Remarque

La figure ci-dessous montre quelques résultats obtenus par la méthode ridge en fonction de la valeur de la pénalité λ sur l'exemple de la régression polynomiale (toujours pour pouvoir représenter les résultats dans un graphique 2D mais le principe est le même dans le cas linéaire multiple).



$$\begin{pmatrix} \dots & x_1^p \\ & \vdots \\ & x_2^p \\ & \vdots \\ & x_m^p \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{pmatrix} + \begin{pmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_m \end{pmatrix}$$

Dans ces 3 exemples, le polynôme est d'ordre 10. Il est régularisé par le coeff λ .

3.3.2 Régression LASSO

La régression ridge permet de contourner les problèmes de colinéarité même en présence d'un nombre important de variables explicatives ou prédicteurs ($p > n$). La principale faiblesse de cette méthode est cependant liée à la difficulté d'interprétation. Sans sélection, toutes les variables sont concernées dans le modèle : elles ont une valeur non-nulle et on ne peut pas se ramener au problème posé au début de Section 3.2.

Pour comprendre l'équivalence entre sélectionner explicitement des variables, comme dans Section 3.2, et sélectionner des variables en ne considérant que les $|\beta_i| > 0$, imaginons que l'on ai 4 variables $\{1, 2, 3, 4\}$ et que les deux variables sélectionnées soient $(1) = 1$ et $(2) = 3$. Alors on a :

The image shows a handwritten matrix equation representing Lasso regression with variable selection. The equation is:

$$\begin{pmatrix} 1 & x_1^{(1)} & x_1^{(2)} \\ 1 & x_2^{(1)} & x_2^{(2)} \\ \vdots & \vdots & \vdots \\ 1 & x_m^{(1)} & x_m^{(2)} \end{pmatrix} \begin{pmatrix} B_0 \\ B_{(1)} \\ B_{(2)} \end{pmatrix} = \begin{pmatrix} 1 & x_1^1 & x_1^2 & x_1^3 & x_1^4 \\ 1 & x_2^1 & x_2^2 & x_2^3 & x_2^4 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_m^1 & x_m^2 & x_m^3 & x_m^4 \end{pmatrix} \begin{pmatrix} B_0 \\ B_1 \\ 0 \\ B_3 \\ 0 \end{pmatrix}$$

avec $|B_1| > 0$ et $|B_3| > 0$

D'autres approches par pénalisation permettent une sélection, c'est le cas de la régression Lasso.

3.3.2 Régression LASSO

Modèle et estimation

La méthode Lasso (Tibshirani, 1996) correspond à la minimisation d'un critère des moindres carrés avec une pénalité de type L_1 (et non L_2 comme dans la régression ridge). Soit $||\beta||_1 = \sum_{j=1}^p |\beta_j|$.

L'estimateur Lasso de β dans le modèle $\mathbf{Y} = \tilde{\mathbf{X}}\tilde{\beta} + \epsilon$ est alors défini par :

$$\hat{\beta}_{LASSO} = \arg \min_{\beta \in \mathbb{R}^{p+1}} \left(\sum_{i=1}^n (Y_i - \sum_{j=0}^p X_i^{(j)} \beta_j)^2 + \lambda \sum_{j=1}^p |\beta_j| \right)$$

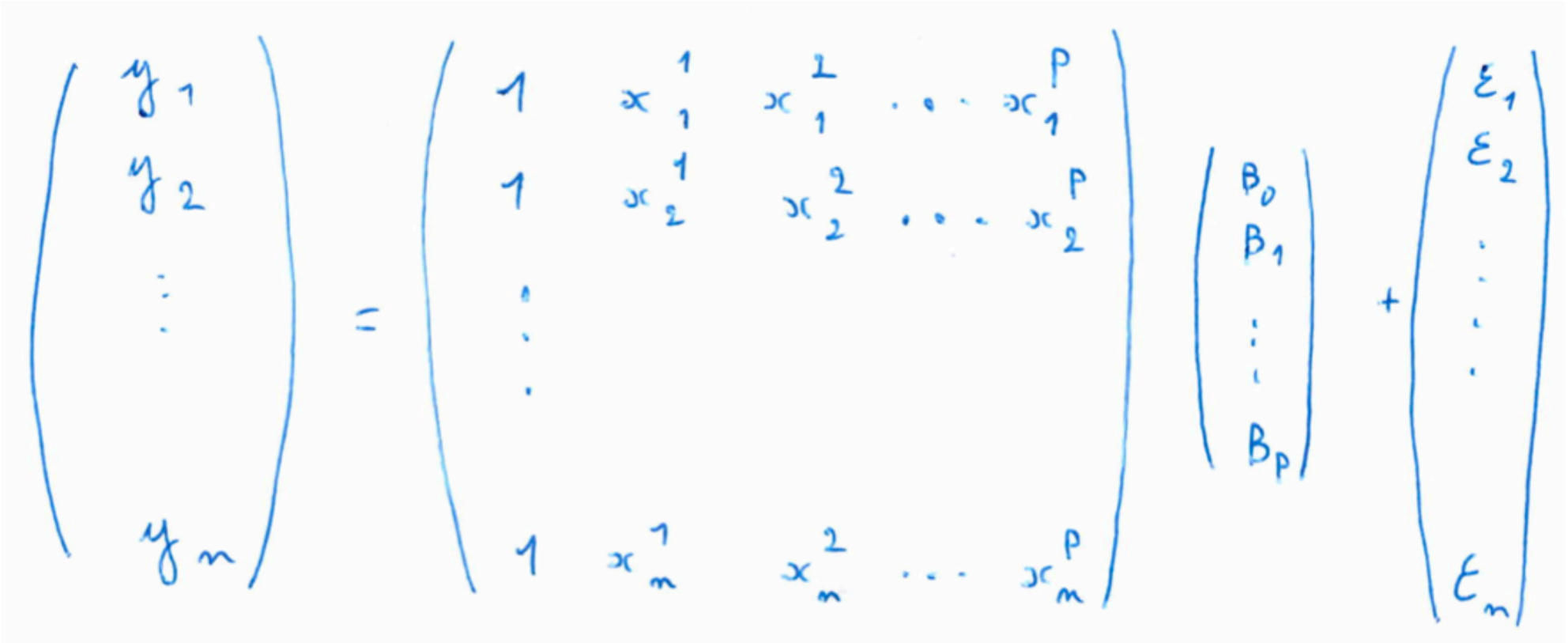
où λ est un paramètre positif. On peut montrer que ceci équivaut au problème de minimisation suivant

$$\hat{\beta}_{LASSO} = \arg \min_{\beta \in \mathbb{R}^p, ||\beta||_1 < t} ||\mathbf{Y} - \mathbf{X}\beta||^2$$

pour un t convenablement choisi. Comme dans le cas de la régression ridge, le paramètre λ est un paramètre de régularisation :

- Si $\lambda = 0$, on retrouve l'estimateur des moindres carrés.
- Si λ tend vers l'infini, on annule tous les $\hat{\beta}_j, j = 1, \dots, p$.

La solution obtenue est dite parcimonieuse (sparse en anglais), car elle comporte des coefficients nuls.



The image shows a handwritten matrix equation representing the Lasso regression model. It is written as follows:

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} 1 & x_1^1 & x_1^2 & \dots & x_1^p \\ 1 & x_2^1 & x_2^2 & \dots & x_2^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n^1 & x_n^2 & \dots & x_n^p \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{pmatrix} + \begin{pmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{pmatrix}$$

3.3.3 Régression Elastic Net

La méthode Elastic Net permet de combiner la régression ridge et la régression Lasso, en introduisant les deux types de pénalités simultanément.

Le critère à minimiser est :

$$\hat{\beta}_{E.N.} = \arg \min_{\beta \in \mathbb{R}^{p+1}} \left(\sum_{i=1}^n (Y_i - \sum_{j=0}^p X_i^{(j)} \beta_j)^2 + \lambda \left(\alpha \sum_{j=1}^p |\beta_j| + (1 - \alpha) \sum_{j=1}^p \beta_j^2 \right) \right)$$

— Pour $\alpha = 1$, on retrouve la méthode LASSO.

— Pour $\alpha = 0$, on retrouve la régression ridge

Il y a dans ce dernier cas deux paramètres à optimiser par validation croisée.

Plus de params. à régler que dans LASSO, mais
fonctionne mieux algorithmiquement.

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} 1 & x_1^1 & x_1^2 & \dots & x_1^p \\ 1 & x_2^1 & x_2^2 & \dots & x_2^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n^1 & x_n^2 & \dots & x_n^p \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{pmatrix} + \begin{pmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{pmatrix}$$

Considérons la formule générique optimisée dans la section précédente :

$$\hat{\beta} = \arg \min_{\beta \in \mathbb{R}^{p+1}} \left(\sum_{i=1}^n (Y_i - \sum_{j=0}^p X_i^{(j)} \beta_j)^2 + \lambda R(\beta_1, \dots, \beta_p) \right)$$

où R est la fonction de pénalisation de β qui regularise le problème d'optimisation.

Trois méthodes de validation croisée (cross-validation) pour valider le choix du paramètre λ et éventuellement de α sont largement utilisées en apprentissage automatique (pas seulement en regression linéaire).

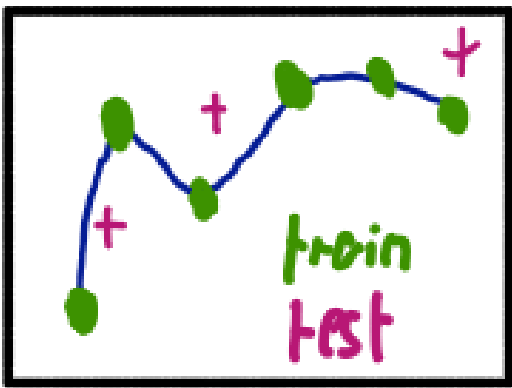
→ Détection du surapprentissage

3.4.1 Subdivision des observations en deux ensembles de données

La méthode élémentaire est de subdiviser les n observations en deux sous ensembles d'observations :

- Les données d'apprentissage.
- les données de validation.

Les données seront idéalement séparées de manière aléatoire, par exemple $i = 1, \dots, n_1$ pour les données d'apprentissage et $i = n_1 + 1, \dots, n$ pour les données de validation.



→ ⚠ Surreprésentation d'un type sur un des sets.
(différences H/F ...)
↳ pour y pallier, ↪

Training set / validation set.

70	/	30
80	/	20

Train

Validation/Test

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix} = \begin{pmatrix} 1 & x_1^1 & x_1^2 & \dots & x_1^p \\ 1 & x_2^1 & x_2^2 & \dots & x_2^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m^1 & x_m^2 & \dots & x_m^p \end{pmatrix} \begin{pmatrix} B_0 \\ B_1 \\ \vdots \\ B_p \end{pmatrix} + \begin{pmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_m \end{pmatrix}$$

4 apprentissages,
4 tests. $K = 4$.

3.4.2 K-folds

Afin de quantifier la stabilité de l'estimation des β_j en fonction des données il est intéressant de reproduire plusieurs fois le test de séparation de données en jeu d'apprentissage et jeu d'estimation.

La méthode la plus simple est celle dite des K-folds. Elle consiste à subdiviser les n observations $(Y_i, \mathbf{X}_i)$ en K jeux de données de taille similaires δ_k , *i.e.* avec δ_k proche de n/K . Pour simplifier les notations, on suppose ici que $\delta_k = n/K$ est entier.

La méthode d'apprentissage-validation decrite dans la sous-section précédente est alors effectuée K fois, avec pour l'itération k :

- Les données d'apprentissage $(Y_i, \mathbf{X}_i)$, $i = 1, \dots, (k - 1)\delta_k, k\delta_k + 1, \dots, n$ sont utilisées pour estimer les β_j^k .
- Les données de validation $(Y_i, \mathbf{X}_i)$, $i = (k - 1)\delta_k + 1, \dots, k\delta_k$. sont utilisées pour calculer e_{split}^k .

$K > 1$ estimation de l'erreur e_{split}^k et des paramètres β_j^k sont alors effectués. Ceci permet d'en mesurer l'erreur de manière plus robuste qu'avec $K = 1$. De plus cela permet de quantifier la variabilité sur l'estimation des β_j : On peut simplement en calculer leur moyenne et écart type. Si une stratégie de sélection de modèle a été effectuée, on peut aussi étudier quels sont les β_j systématiquement sélectionnés et quels sont ceux qui le sont moins.

Critère : si trop de variabilité sur les testsets, il y a pb.
Coûte cher.

T
V

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix} = \begin{pmatrix} 1 & x_1^1 & x_1^2 & \dots & x_1^p \\ 1 & x_2^1 & x_2^2 & \dots & x_2^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m^1 & x_m^2 & \dots & x_m^p \end{pmatrix} \begin{pmatrix} B_0 \\ B_1 \\ \vdots \\ B_p \end{pmatrix} + \begin{pmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{pmatrix}$$

puis

T
V
T

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix} = \begin{pmatrix} 1 & x_1^1 & x_1^2 & \dots & x_1^p \\ 1 & x_2^1 & x_2^2 & \dots & x_2^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m^1 & x_m^2 & \dots & x_m^p \end{pmatrix} \begin{pmatrix} B_0 \\ B_1 \\ \vdots \\ B_p \end{pmatrix} + \begin{pmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{pmatrix}$$

puis

T
V
T

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix} = \begin{pmatrix} 1 & x_1^1 & x_1^2 & \dots & x_1^p \\ 1 & x_2^1 & x_2^2 & \dots & x_2^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m^1 & x_m^2 & \dots & x_m^p \end{pmatrix} \begin{pmatrix} B_0 \\ B_1 \\ \vdots \\ B_p \end{pmatrix} + \begin{pmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{pmatrix}$$

puis

V
T

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix} = \begin{pmatrix} 1 & x_1^1 & x_1^2 & \dots & x_1^p \\ 1 & x_2^1 & x_2^2 & \dots & x_2^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m^1 & x_m^2 & \dots & x_m^p \end{pmatrix} \begin{pmatrix} B_0 \\ B_1 \\ \vdots \\ B_p \end{pmatrix} + \begin{pmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{pmatrix}$$

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} 1 & x_1^1 & x_1^2 & \dots & x_1^p \\ 1 & x_2^1 & x_2^2 & \dots & x_2^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n^1 & x_n^2 & \dots & x_n^p \end{pmatrix} \begin{pmatrix} B_0 \\ B_1 \\ \vdots \\ B_p \end{pmatrix} + \begin{pmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{pmatrix}$$

puis

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} 1 & x_1^1 & x_1^2 & \dots & x_1^p \\ 1 & x_2^1 & x_2^2 & \dots & x_2^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n^1 & x_n^2 & \dots & x_n^p \end{pmatrix} \begin{pmatrix} B_0 \\ B_1 \\ \vdots \\ B_p \end{pmatrix} + \begin{pmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{pmatrix}$$

puis

⋮

puis

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} 1 & x_1^1 & x_1^2 & \dots & x_1^p \\ 1 & x_2^1 & x_2^2 & \dots & x_2^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n^1 & x_n^2 & \dots & x_n^p \end{pmatrix} \begin{pmatrix} B_0 \\ B_1 \\ \vdots \\ B_p \end{pmatrix} + \begin{pmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{pmatrix}$$

3.4.3 Leave-one-out

La méthode de validation croisée dite *Leave-one-out* est extrêmement populaire en apprentissage automatique et est équivalent aux K-folds avec $K = n$. A chaque itération, l'apprentissage est effectué en enlevant une observation du jeu de données et la validation est faite sur cette observation. Cette méthode est plus lente que les K-folds en particulier quand n est grand, mais est la plus robuste et recommandée quand n est petit.

Seulement si l'apprentissage est pas cher.
Très utile si il y a peu de données.